

Paterni Ponašanja

-Strategy patern

- Strategy patern omogućava klijentu izbor jednog od algoritama iz familije algoritama.
 - Implementacije algoritama su neovisne od klijenata koji ih koriste.
 - U našem primjeru ovaj pattern bismo mogli iskoristiti kod klase Popust.
 - Različiti korisnici mogu imati različite vrste popusta. Klasa rezervacija ne bi trebala sama provjeravati koji algoritam za izračun popusta da primjeni. Umjesto toga mogli bismo da za svaki način izračuna popusta napravimo posebnu klasu.
 - Time se omogućava lako dodavanje novih vrsta popusta. Bez potrebe za izmjenom klase Rezervacija.
 - Postoje tri načina prosljeđivanja algoritma/strategije.
 1. Parameter injection
 2. Method injection
 3. Konstruktor injection
- Za naš sistem najbolje je koristiti Method injection ako se određeni algoritam koristi duže vremena(jer on pamti taj algoritam u svome privatnom atributu).

-State patern

- State patern omogućava objektu da izmijeni svoje ponašanje na osnovu promjene njegovog unutrašnjeg stanja.
- Umjesto da u klasi imamo veliki broj if-else blokova za provjeru stanja, bolje je da svako stanje bude posebna klasa. Na osnovu toga koje stanje je aktivno klase se ponašaju na određeni način.
- U našem sistemu ovaj patern je najbolje primjeniti na klasu rezervacija.
- Naime, rezervacija se treba ponašati drugačije u zavisnosti od trenutnog stanja.
- Možemo napraviti interfejs StanjeRezervacije, a zatim konkretne klase kao što su KreiranaRezervacija, PotvrđenaRezervacija, OtkazanaRezervacija i ZavršenaRezervacija.
- Sistem postaje fleksibilniji jer se novo stanje može dodati kao nova klasa, bez velikih izmjena postojećeg koda.

-Template patern

- Omogućava izdvajanje određenih koraka algoritma u odvojene podklase.
- Struktura algoritma se ne mijenja - specifični dijelovi operacija se izdvajaju i njih podklase mogu implementirati na različite načine.
- U našem sistemu pomenuti patern najbolje je primjeniti na Izvještaj. Npr. Zaposlenikov izvještaj se razlikuje od vlasnikovog u nekim stvarima.
- Template Method patern možemo iskoristiti tako što ćemo u baznoj klasi Izvještaj definisati kostur kreiranja izvještaja kroz metodu, npr. kreirajIzvjestaj(). Ta metoda određuje tačan redoslijed koraka i ne mijenja se u podklasama. Konkretne klase, kao što su IzvjestajZaposlenika i IzvjestajVlasnika, implementiraju samo one dijelove koji se razlikuju. Time se izbjegava dupliranje koda i osigurava da svi izvještaji prate isti osnovni proces kreiranja.

-Observer patern

- Uspostavlja relaciju između objekata, kada jedan objekat promijeni stanje drugi zainteresovani objekti se automatski obavještavaju.
- Posmatrači se pretplate na određeni objekat i automatski budu obaviješteni kada se desi promjena.
- U našem sistemu, ovaj patern bismo mogli primjeniti kod notifikacija koje su vezane za rezervaciju. Kada se rezervacija potvrdi, otkaže ili promjeni da obavijest o tome dobije više dijelova sistema.
- Rezervacija predstavlja subjekt(tj. Onaj ko inicira obavijest), a možemo napraviti više Observer klasa, npr. EmailNotifikacija, SMSNotifikacija,...
- Kada se stanje rezervacije promijeni svi prijavljeni posmatrači dobijaju obavještenje.
- U C# ovo se implementira preko eventa i delegata. Event je ustvari lista registrovanih posmatrača, lista se tada ne vidi direktno u kodu.
- Rezervacija ne mora znati ko sve prima obavijest, nego samo objavi promjenu.

-Iterator patern

- Iterator patern omogućava pristup elementima kolekcije bez poznavanja kako je kolekcija strukturirana.
- Iteratori implementiraju različite algoritme navigacije kroz kolekciju podataka.
- U našem sistemu ovaj patern možemo iskoristiti za prolazak kroz listu termina.
- Termini mogu biti organizovani po danima, sportovima, terenima ili vremenskim intervalima.
- Iterator omogućava redom prolazak kroz sve termine.

-Prednost je u tome što se u slučaju promjene kolekcije podataka u kojoj čuvamo termine ostatak sistema ne mora mijenjati.

-Visitor patern

-Glavna svrha je odvajanje algoritama i operacija od strukture objekata nad kojima se te operacije izvršavaju.

-Omogućava dodavanje novih funkcionalnosti i operacija u postojeće klase, a da se te klase uopće ne moraju mijenjati ili ponovo kompajlirati.

-Definiše novu operaciju koja se izvršava nad objektima bez mijenjanja njihovih klasa.

-U našem sistemu mogli bi ovaj patern primjeniti na usluge za različite klase(vrste) terena.

-Kad god imamo potrebu da za sve terene uradimo istu operaciju, da ne bi ulazili u svaku klasu i pisali sve metode koje nam trebaju. Jednostavno, u svaku klasu dodamo funkciju Accept().

-Tereni preko funkcije Accept() puštaju visitora da pristupi podacima i odradi operacije.

-Command patern

-Koristi se za enkapsuliranje svih informacija potrebnih za odgođeno izvođenje akcije ili pokretanje događaja.

-Glavna svrha je pretvaranje zahtjeva ili operacije u samostalan objekat koji sadrži sve informacije o tom zahtjevu.

-Omogućava da se operacije izvršavaju, odgađaju, stavljaju u red čekanja ili poništavaju.

-U našem sistemu ovaj patern bismo mogli primijeniti kod akcija koje korisnik ili admin izvršava nad rezervacijama.

-Svaka od akcija nad rezervacijama postaje posebna klasa, koja implementira zajednički interface sa metodom execute.

-Objekat koji inicira akciju(Invoker) je u potpunosti nezavisan od objekta koji izvršava logiku iza te akcije(Receiver).

-Sve komande možemo pratiti u nekoj listi, to nam omogućava poništavanje komandi.

-Također, pošto su komande sada objekti, možemo ih stavljati u red čekanja.